# Demostración 1: Proyecto Grupal 01

Modelado de Interconnect para el análisis de desempeño de Protocolos de Coherencia HW/SW

CE 4302 — Arquitectura de Computadores II
Instituto Tecnológico de Costa Rica

Randall Bryan Bolaños Lopez - *Arquitecto de Hardware (SystemC/TLM)*
Andres Molina Redondo - *Ingeniero de Protocolos de Coherencia*
Christopher Rodríguez Cordero - *Ingeniero de Software y Workloads*

9 de abril del 2026

## 1. Conformación de Equipos de Trabajo y Roles

El equipo de trabajo está conformado por tres personas, distribuyendo las responsabilidades técnicas del proyecto de la siguiente manera:

- **Randall Bryan Bolaños Lopez:** Arquitecto de Hardware (SystemC/TLM) - Encargado del diseño del Interconnect y la lógica de arbitraje.
- **Andres Molina Redondo:** Ingeniero de Protocolos de Coherencia - Encargado de la implementación de los protocolos Write-Invalidate (MSI) y Write-Update (Firefly).
- **Christopher Rodríguez Cordero:** Ingeniero de Software y Workloads - Responsable de la generación de *memory traces* y el desarrollo de los algoritmos paralelos en el entorno de pruebas.

## 2. Manejo del Repositorio GIT

Para asegurar un flujo de trabajo ordenado y colaborativo, el equipo ha acordado con los profesores el siguiente modelo de gestión para el repositorio Git:

- **Plataforma:** GitHub / GitLab.
- **Estrategia de Ramas (Branching):** Se utilizará un enfoque basado en *Feature Branches*. Existirá una rama `main` para código funcional y estable, una rama `develop` para la integración, y ramas individuales (ej. `feat/interconnect`, `feat/msi-protocol`) para el desarrollo de componentes específicos.
- **Revisiones (Code Review):** La integración de código a la rama `develop` requerirá un *Pull Request* revisado por al menos un miembro distinto al autor original.

# 3. Investigación de Workloads

Para evaluar el desempeño y los cambios en el tráfico del sistema bajo los protocolos de coherencia implementados, se han seleccionado y modelado los siguientes dos escenarios paralelos. En ambos casos, se operará sobre la arquitectura base estipulada de **4 Elementos de Procesamiento (PEs)** operando en simultáneo:

## 3.1. Multiplicación de Matrices Bloqueada (Alta Contención)

**Descripción:** Este algoritmo divide matrices de gran tamaño en sub-bloques más pequeños para que cada uno de los 4 PEs realice cálculos locales en paralelo antes de actualizar la matriz resultante compartida.
**Justificación Arquitectónica:** Este *workload* es propenso a generar un alto índice de *False Sharing* (falso compartimiento) si el tamaño de los bloques no está perfectamente alineado con las líneas de caché L1. Esto provocará alta contención en el bus, siendo un escenario ideal para estresar y comparar la cantidad de invalidaciones del protocolo MSI frente a las actualizaciones del protocolo de la familia Write-Broadcast/Update.

## 3.2. Pipeline de Procesamiento de Criptografía / Red (Productor-Consumidor)

**Descripción:** Se simula una tubería de procesamiento de datos en etapas a través de los 4 PEs. Por ejemplo, el PE0 recibe y desencripta un paquete de red, luego lo escribe en una cola compartida; el PE1 lo lee y realiza inspección profunda; el PE2 lo comprime y finalmente el PE3 lo empaqueta para su envío.
**Justificación Arquitectónica:** Este algoritmo representa un patrón migratorio unidireccional y de Productor-Consumidor clásico. Es fundamental para observar cómo los diferentes protocolos manejan las transferencias de lectura/escritura secuenciales a lo largo del pipeline. Se espera que un protocolo Write-Update (como Firefly) muestre una ventaja de rendimiento al actualizar proactivamente las cachés de los PEs consumidores de forma escalonada, reduciendo la latencia general frente a un protocolo Write-Invalidate.